

# Frameworks Web: Flask e React

Arquiteturas de Renderização e Interfaces Reativas

Prof.<sup>a</sup> Catarina Silva

Ano Letivo 2025/2026 – 2º Semestre

# Definição de Framework Web

---

- Uma **Framework Web** constitui uma abstração tecnológica que disponibiliza uma estrutura genérica e funcionalidade pré-definida para o desenvolvimento de aplicações.
- Estas plataformas fornecem mecanismos padronizados para a gestão de protocolos de comunicação, persistência de dados e segurança, mitigando a necessidade de implementação manual de funcionalidades de baixo nível.
- A adoção de frameworks promove o rigor arquitetural, a interoperabilidade de sistemas e a manutenibilidade do código fonte.

# Arquitetura Flask: Mapeamento de Rotas e Views

---

- A infraestrutura do Flask assenta no conceito de **Roteamento**, que estabelece a correspondência entre um Uniform Resource Locator (URL) e uma sub-rotina específica em Python.
- A **View** é a entidade lógica responsável pelo processamento da requisição HTTP, execução da lógica de negócio e subsequente geração da resposta.
- Através de **decoradores de função**, o Flask implementa uma interface declarativa para a definição de pontos de acesso (endpoints), otimizando a legibilidade estrutural.

# Motores de Templates e Renderização Dinâmica

---

- Um **Motor de Templates** viabiliza a construção de documentos HTML que integram marcadores de dados dinâmicos e lógica de controlo.
- O **Jinja2** permite a execução de estruturas iterativas e condicionais em tempo de renderização, permitindo a injeção de dados processados pelo servidor no documento final.
- O processo culmina na geração de um documento estático personalizado para o cliente, garantindo a separação entre a camada de dados e a camada de apresentação.

# Processamento de Dados e o Método HTTP POST

---

- A submissão de dados via formulários deve priorizar o método **POST**, assegurando que a informação não seja exposta na estrutura da URL ou no histórico de navegação.
- O objeto request providencia o acesso programático aos dados submetidos, permitindo a validação de integridade e o armazenamento seguro de credenciais ou metadados.
- A implementação do padrão *Post/Redirect/Get* é imperativa para prevenir a submissão duplicada de dados aquando da atualização da interface pelo utilizador.

# Mitigação de Ataques Cross-Site Request Forgery (CSRF)

---

- O **CSRF** é uma vulnerabilidade de segurança que permite a execução de ações não autorizadas em nome de um utilizador autenticado através de requisições maliciosas de terceiros.
- A defesa contra esta vulnerabilidade baseia-se na utilização de um **Token de Sincronização**: um identificador único, pseudo-aleatório e de curta duração associado à sessão.
- A validação obrigatória deste token no servidor garante a origem fidedigna da requisição e a autenticidade da interação do utilizador.

# Desenvolvimento Declarativo e Reatividade

---

- O **React** introduz um paradigma declarativo, onde o estado da interface é uma função direta dos dados da aplicação.
- Ao contrário da manipulação imperativa do Document Object Model (DOM), o programador define a representação visual para cada estado possível da aplicação.
- A biblioteca gere de forma autónoma as mutações necessárias na interface, minimizando operações de renderização redundantes e aumentando a eficiência computacional.

# Arquitetura Interna: Virtual DOM e Reconciliação

---

- O **Virtual DOM** é uma representação leve, em memória, da estrutura real do Document Object Model.
- O processo de **Reconciliação** consiste no algoritmo de comparação (*diffing*) entre a árvore virtual atual e a árvore resultante de uma alteração de estado.
- Esta arquitetura permite que o React identifique o conjunto mínimo de alterações necessárias, aplicando-as de forma otimizada ao DOM real para maximizar a performance.



# Abstração de Interfaces: Componentes e Sintaxe JSX

---

- A arquitetura React é fundamentada em **Componentes**: unidades modulares, encapsuladas e reutilizáveis que definem fragmentos da interface de utilizador.
- O **JSX** (JavaScript XML) é uma extensão sintática que permite a descrição da estrutura documental integrada na lógica de programação JavaScript.
- Esta integração facilita a construção de interfaces complexas através da composição de componentes atómicos, promovendo a consistência visual e lógica.

# O Pipeline de Build: Transpilação e Empacotamento

---

- Os navegadores web modernos não possuem suporte nativo para a sintaxe JSX ou funcionalidades experimentais do ECMAScript.
- O processo de **Transpilação** (através de ferramentas como o Babel) converte o código fonte moderno numa versão de JavaScript amplamente compatível.
- Ferramentas de **Bundling** (como Vite ou Webpack) otimizam a aplicação, consolidando múltiplos ficheiros e dependências em pacotes estáticos eficientes para produção.

# Persistência Local: O Hook `useState`

---

- O **Estado** (*State*) constitui a representação da informação volátil e reativa de um componente num dado instante temporal.
- Através do Hook **`useState`**, o React providencia um mecanismo para a monitorização de alterações nestes dados.
- A modificação do estado despoleta automaticamente o ciclo de vida de renderização, assegurando a sincronia entre a lógica da aplicação e a representação visual.

# Arquitetura de Estado: Local vs. Global

---

- O estado local é confinado ao domínio de um único componente, sendo ideal para gerir interações isoladas (ex: estados de formulário).
- Em sistemas complexos, surge a necessidade de um **Estado Global**, permitindo a partilha de informação transversal a múltiplos ramos da árvore de componentes.
- Soluções como a Context API ou Redux oferecem mecanismos para evitar o *prop drilling*, garantindo uma gestão centralizada e previsível dos dados da aplicação.

# Gestão de Efeitos Secundários: O Hook `useEffect`

---

- Operações que transcendem o domínio da renderização pura são classificadas como **Efeitos Secundários** (Side Effects).
- O Hook **`useEffect`** permite a execução controlada de lógica assíncrona, manipulação direta de APIs globais ou subscrição de serviços externos.
- A definição de uma matriz de dependências assegura que o efeito seja executado apenas quando os dados relevantes sofrem mutações, otimizando o ciclo de processamento.

# Princípios da Arquitetura RESTful

---

- O modelo **REST** (Representational State Transfer) define um conjunto de restrições para a criação de serviços web escaláveis e independentes.
- Entre os seus pilares destacam-se a **Interface Uniforme**, a utilização de métodos HTTP padronizados e a natureza **Stateless** (sem estado) das requisições.
- Esta padronização assegura que o servidor e o cliente possam evoluir de forma independente, utilizando o formato JSON como linguagem comum de comunicação.

# Interoperabilidade e Segurança: Cross-Origin Resource Sharing (CORS)

---

- Por predefinição, os navegadores aplicam a **Same-Origin Policy**, impedindo que um cliente web acesse recursos de um servidor num domínio ou porta diferente.
- O **CORS** é um mecanismo baseado em cabeçalhos HTTP que permite ao servidor declarar quais as origens autorizadas a consumir os seus recursos.
- A configuração correta de políticas CORS no Flask é crítica para viabilizar a comunicação legítima com o cliente React sem comprometer a segurança do ecossistema.

# Integração de Sistemas: O Formato de Intercâmbio JSON

---

- A interoperabilidade entre o servidor Flask e o cliente React é viabilizada através da disponibilização de uma **Application Programming Interface** (API).
- O formato **JSON** (JavaScript Object Notation) é adotado como padrão de intercâmbio de dados devido à sua natureza agnóstica a linguagens e reduzida sobrecarga de processamento.
- Esta arquitetura desacoplada permite a criação de **Single Page Applications** (SPA), onde o estado da interface evolui dinamicamente através de requisições assíncronas.



# Hierarquia de Dados: Propriedades e Fluxo Unidirecional

---

- O React implementa um **fluxo de dados unidirecional**, onde a informação flui estritamente de componentes ancestrais para componentes descendentes.
- As **Props** (Propriedades) são argumentos de entrada imutáveis que permitem a configuração e parametrização de componentes filhos.
- Esta previsibilidade na propagação de dados facilita a depuração e o rastreamento do estado global da aplicação em arquiteturas de elevada complexidade.

# Síntese Arquitetural e Boas Práticas

---

- O ecossistema de desenvolvimento web moderno exige a separação clara entre a **lógica de persistência** (Backend) e a **lógica de apresentação** (Frontend).
- A utilização de frameworks robustas e padrões de segurança verificados é mandatória para a entrega de soluções de software de nível empresarial.
- O domínio destes conceitos teóricos permite a construção de sistemas escaláveis, performantes e em conformidade com os requisitos industriais contemporâneos.